Самостоятельная работа студента (CPC / SIW 2)

Сравнение моделей параллельного программирования OpenMP и MPI

Эксперимент на примере одномерного уравнения теплопроводности: реализации на OpenMP, MPI и гибридная MPI + OpenMP

Выполнил: _____

Проверил: _____

2026

1. Цель и постановка задачи

Задание SIW 2 — не пересказ двух стандартов, а эксперимент, предметом которого являются две модели параллельного программирования. Цель: понять, какие допущения о памяти машины делает каждая модель, к чему эти допущения приводят для программиста, и подтвердить выводы собственными измерениями. Для этого один и тот же алгоритм реализован трижды — на OpenMP, на MPI и в виде гибридной программы MPI + OpenMP, — после чего проверена корректность, измерены время, ускорение и эффективность, определены параметры канала связи MPI (задержка ts и пропускная способность B), исследованы деления ресурсов в гибридной программе и расход памяти.

Все числа в работе получены автором на одной машине (описана в разделе 3.2) командами, приведёнными в разделе 3.3 и в репозитории (скрипты scripts/run_all.sh, scripts/run_cache.sh). Исходные тексты — в приложении А. Там, где результат оказался неожиданным или слабым (например, ускорение ограничено пропускной способностью памяти), это прямо указано: честный анализ ценнее «красивого» графика.

2. Теоретическая часть

2.1. Модель памяти — главное различие

OpenMP [5] предполагает единое адресное пространство, видимое каждому потоку: любой поток может прочитать или записать любую переменную без явной пересылки. MPI [4] предполагает, что у каждого процесса собственное, закрытое адресное пространство, а любой обмен данными — явный (отправка и приём сообщений либо коллективная операция).

Это единственное различие определяет почти всё остальное. Если данные общие, то обмен «бесплатен» синтаксически, но программист отвечает за порядок доступа (гонки данных, барьеры, блокировки), а стоимость скрыта в аппаратуре — в когерентности кэшей. Если данные раздельны, то гонок между процессами нет, но каждая пересылка видна в тексте программы и имеет цену в виде сообщения; зато модель естественно переносится на кластер, где общей памяти нет вообще [1, 2].

2.2. Модель исполнения OpenMP: fork–join

Программа стартует одним главным (master) потоком. На входе в параллельную область (#pragma omp parallel) создаётся команда потоков, в конструкциях разделения работы (omp for, sections) итерации распределяются между потоками, на выходе из области происходит неявный барьер, после чего снова работает один поток. Важное свойство: если директивы проигнорировать (компилировать без -fopenmp), получится корректная последовательная программа. Это делает возможным инкрементальное распараллеливание — по одному циклу за раз.

2.3. Модель исполнения MPI: SPMD

В MPI каждый процесс исполняет один и тот же исходный текст от первой строки до последней (single program, multiple data); различие поведения задаётся рангом (rank) процесса внутри коммуникатора (communicator) — группы процессов, между которыми разрешён обмен. MPI_COMM_WORLD содержит все процессы. Параллелизм в MPI нельзя «выключить»: программа с первой строки написана как параллельная, поэтому инкрементальный переход от последовательной версии невозможен — приходится заново проектировать декомпозицию данных.

2.4. Синхронизация и обмен данными

В OpenMP данные по умолчанию общие, и программист обязан объявить, что приватно (private, firstprivate, reduction), и упорядочить доступ с помощью барьеров (barrier), критических

секций (critical), атомарных операций (atomic) и замков (omp_lock). В MPI ничего не разделяется: программист задаёт каждую передачу — двухточечными операциями (MPI_Send/Recv, MPI_Sendrecv) или коллективными (MPI_Bcast, MPI_Reduce, MPI_Gather). В нашей задаче граничный обмен — это две операции MPI_Sendrecv на шаг; в OpenMP тот же обмен вообще не пишется, а роль синхронизации играет барьер между шагами.

2.5. Модели стоимости

В модели с общей памятью доминируют синхронизация (барьеры, замки) и трафик когерентности: запись в разделяемую строку кэша заставляет остальные ядра сбрасывать свои копии. Предельный случай — ложное разделение (false sharing): потоки пишут в разные переменные, лежащие в одной 64-байтной строке кэша, и ядра «пинг-понгом» перебрасывают строку друг другу. В распределённой модели доминирует сообщение, время которого описывается формулой

$$T_comm = t_s + M / B \qquad (1)$$

где t_s — латентность, M — длина сообщения, B — пропускная способность канала. Для коротких сообщений (M < t_s·B) время определяется латентностью, и выгодно объединять мелкие обмены в крупные; для длинных — пропускной способностью. Какая модель применима к циклу, определяется тем, что в нём происходит: цикл, где потоки читают общий массив и пишут в непересекающиеся места, подчиняется модели общей памяти (стоимость — барьер в конце и конкуренция за шину памяти); цикл, после которого данные нужно доставить в другое адресное пространство, подчиняется формуле (1). В нашем шаге схемы вычисление обновляет N/p узлов (чистая работа с памятью), а обмен — всего два числа типа double на соседа, т. е. M = 8 байт ≪ t_s·B: обмен целиком определяется латентностью.

2.6. Критерии сравнения

В аналитической части (раздел 4) модели сравниваются по семи критериям: (1) модель памяти и видимость данных; (2) механизм и стоимость коммуникации; (3) предпочитаемая гранулярность параллелизма; (4) предел масштабируемости и его причина; (5) возможность инкрементального распараллеливания; (6) сложность отладки и классы дефектов; (7) переносимость между машинами и компиляторами.

2.7. Гибридная модель MPI + OpenMP

Современный кластер — это множество узлов, каждый из которых содержит один-два сокета с многоядерными процессорами. Естественное отображение: один MPI-процесс на сокет (или на NUMA-домен), внутри процесса — потоки OpenMP по ядрам сокета. Так число MPI-рангов и число сообщений уменьшаются, снижается дублирование памяти, а память сокета используется потоками совместно. Уровень поддержки потоков задаётся при MPI_Init_thread: MPI_THREAD_SINGLE (потоков нет), FUNNELED (MPI вызывает только главный поток), SERIALIZED (любой поток, но по одному за раз), MULTIPLE (любые потоки одновременно, самая дорогая реализация) [4, 6]. В нашей гибридной программе используется FUNNELED: обмен границами выполняет главный поток, остальные ждут на барьере.

3. Практическая часть

3.1. Алгоритм и реализации

Выбрано одномерное уравнение теплопроводности u_t = a·u_xx, решаемое явной схемой FTCS: u_i^(n+1) = u_i^n + r·(u_(i−1)^n − 2·u_i^n + u_(i+1)^n), r = 0,25 (условие устойчивости r ≤ 0,5). Границы — Дирихле (u = 0). Начальное условие u(x,0) = sin(πx) + 0,5·sin(3πx). Алгоритм содержит и вычисление (трёхточечный шаблон), и обмен между подобластями (граничные узлы соседей).

- Общее ядро. Функция kernel(a, b, lo, hi) из файла core.h одинакова во всех трёх программах (и в последовательном эталоне), поэтому измеряется модель программирования, а не арифметика.
- OpenMP (solver_omp.c): один процесс, массив общий; каждый поток обновляет свой статический блок индексов, между шагами — барьер. Инициализация параллельная (политика «первого касания» — first touch).
- MPI (solver_mpi.c): область разбита на полосы по рангам, у каждого ранга локальный массив с двумя «призрачными» ячейками; на каждом шаге — обмен границами двумя вызовами MPI_Sendrecv.
- Гибрид (solver_mpi.c, -DHYBRID): MPI_Init_thread(FUNNELED); внутри процесса потоки OpenMP считают свой блок; границы обменивает главный поток (#pragma omp master) между барьерами.

3.2. Стенд

Процессор Intel Core i5-13400F (10 ядер, 16 аппаратных потоков), L2 — 10 МиБ, L3 — 20 МиБ, ОЗУ 16 ГиБ (в среде 15,5 ГиБ). Измерения выполнены в WSL2 (Ubuntu 24.04) под Windows 11: gcc 13.3.0, Open MPI 4.1.6. Виртуальная среда показывает 8 «ядер» × 2 потока и один узел NUMA. Следствия, которые важны для интерпретации: (а) сеть отсутствует, MPI использует транспорт через разделяемую память, поэтому t_s и B — характеристики именно внутриузлового обмена, а не кластера; (б) один NUMA-узел — эффект first touch в явном виде измерить нельзя; (в) работающая рядом ОС Windows добавляет шум, особенно при 16 потоках, когда заняты все аппаратные потоки. Эти ограничения указаны и в выводах.

3.3. Команды запуска

Размещение задано явно: каждому MPI-процессу выделяется непрерывный набор из 16/R аппаратных потоков, потоки OpenMP внутри набора раскладываются OMP_PROC_BIND=spread, OMP_PLACES=threads. Фактические привязки проверялись ключом --report-bindings.

```
gcc   -O3 -g -fopenmp solver_omp.c -o solver_omp -lm
mpicc -O3 -g solver_mpi.c -o solver_mpi -lm
mpicc -O3 -g -fopenmp -DHYBRID solver_mpi.c -o solver_hyb -lm
# OpenMP:   p потоков
OMP_NUM_THREADS=p OMP_PLACES=threads OMP_PROC_BIND=spread ./solver_omp N steps reps
# MPI:      R процессов
mpirun -n R --use-hwthread-cpus --map-by ppr:R:node:PE=$((16/R)) --bind-to hwthread ./solver_mpi N steps
reps
# Гибрид:   R процессов x T потоков
OMP_NUM_THREADS=T OMP_PLACES=threads OMP_PROC_BIND=spread \
mpirun -n R --use-hwthread-cpus --map-by ppr:R:node:PE=$((16/R)) --bind-to hwthread --report-bindings
./solver_hyb N steps reps
```

Два режима задачи: (А) N = 2^24 = 16 777 216 узлов, 100 шагов (рабочий набор 256 МиБ — не помещается в кэш, нагрузка на память); (Б) N = 2^18 = 262 144 узлов, 8000 шагов (рабочий набор 4 МиБ — помещается в кэш, нагрузка на синхронизацию). Каждая точка — 6 прогонов (режим А) или 11 прогонов (режим Б), первый отброшен, берётся медиана; в таблицах указан разброс (max − min).

3.4. Проверка корректности

Каждая реализация сравнена с последовательной программой в норме $L\infty = \max_i |u_i - u_i^{serial}|$ на N = 1 000 003 узлах (число намеренно не кратно количеству процессов) после 200 шагов, для p = 1, 2, 4, 8, 16 и для пяти делений гибрида (всего 15 запусков). Во всех случаях получено L∞ = 0.0e+00, т. е. результаты совпали с эталоном побитово. Это ожидаемо: каждый узел вычисляется одним и тем же выражением с одним порядком операций, а редукций (чьи результаты зависят от порядка сложения) в алгоритме нет. Поэтому сравнение именно по норме L∞, а не по равенству, здесь остаётся правильным критерием, но нулевая норма не должна удивлять; для задач с редукциями ожидается значение порядка 1e-16…1e-12.

3.5. Время, ускорение и эффективность

Ускорение и эффективность считаются по формулам (2), где T(1) — время той же реализации при одном потоке или процессе:

$$S(p) = T(1) / T(p), \quad E(p) = S(p) / p \qquad (2)$$

Таблица 1 — Режим А: N = 2^24, 100 шагов (память). Гибрид: 2 процесса × p/2 потоков (при p = 1 — 1×1)

p	OMP T, с	S	E	MPI T, с	S	E	Гибрид T, с	S	E
1	1.177	1.00	1.00	1.177	1.00	1.00	1.170	1.00	1.00
2	1.054	1.12	0.56	1.048	1.12	0.56	1.045	1.12	0.56
4	0.880	1.34	0.33	0.901	1.31	0.33	0.863	1.36	0.34
8	0.760	1.55	0.19	0.830	1.42	0.18	0.756	1.55	0.19
16	0.815	1.44	0.09	0.853	1.38	0.09	0.823	1.42	0.09

Таблица 2 — Режим Б: N = 2^18, 8000 шагов (кэш). Гибрид: 2 процесса × p/2 потоков

p	OMP T, с	S	E	MPI T, с	S	E	Гибрид T, с	S	E
1	0.674	1.00	1.00	0.688	1.00	1.00	0.715	1.00	1.00
2	0.378	1.78	0.89	0.401	1.72	0.86	0.395	1.81	0.90
4	0.215	3.13	0.78	0.210	3.28	0.82	0.236	3.03	0.76
8	0.164	4.11	0.51	0.165	4.18	0.52	0.171	4.19	0.52
16	0.419	1.61	0.10	0.377	1.82	0.11	0.521	1.37	0.09

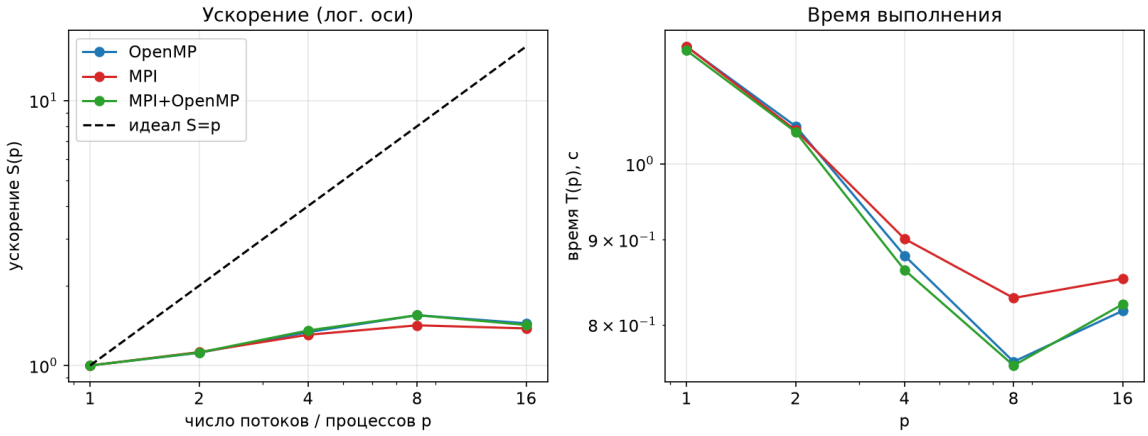


Рис. 1 — Режим А (N = 2^24): ускорение трёх реализаций в логарифмических осях и время выполнения

Режим А. Ускорение быстро упирается в потолок около 1.5: при p = 8 OpenMP даёт S = 1.55, MPI — 1.42, гибрид — 1.55; при p = 16 ускорение даже снижается. Причина — не модель программирования, а пропускная способность памяти: на каждое обновление узла приходится порядка 24 байт трафика (чтение, запись и подгрузка строки при записи), поэтому одна нить уже обрабатывает данные со скоростью ≈ 34 ГБ/с, а при p = 8 — всего ≈ 53 ГБ/с (оценки по формуле 24 Б × N × шагов / T). Три реализации ведут себя практически одинаково — разница порядка 1–9 % (самая заметная при p = 8: OpenMP 0,760 с против MPI 0,830 с при разбросе 0,03 и 0,02 с).

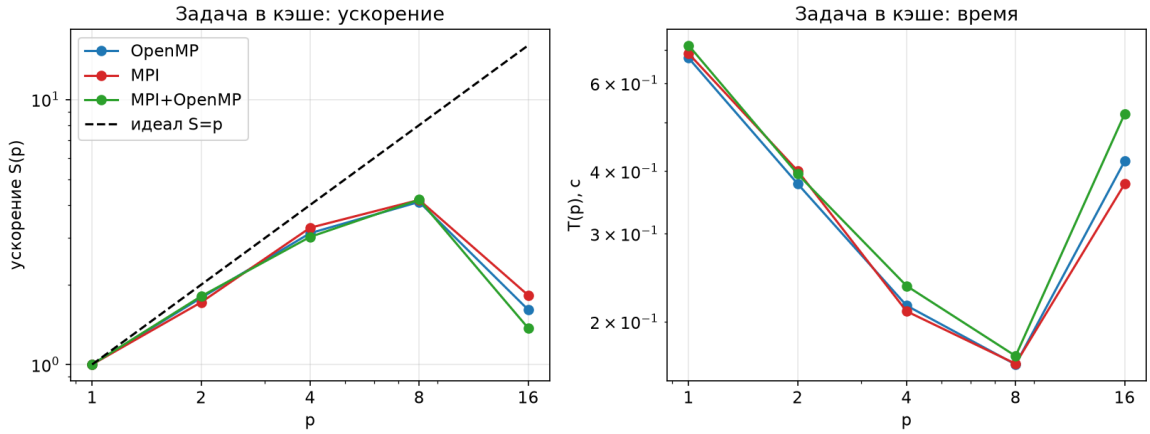


Рис. 2 — Режим Б (N = 2^18, рабочий набор в кэше): ускорение и время

Режим Б. Когда данные помещаются в кэш, узкое место переходит с памяти на синхронизацию, и ускорение заметно лучше: при p = 8 все три реализации дают S ≈ 4.11–4.18 (E ≈ 0,5). При p = 4 эффективность — около 0,8. Снова OpenMP, MPI и гибрид практически неотличимы: стоимость барьера внутри сокета и стоимость двух MPI_Sendrecv по разделяемой памяти оказываются сопоставимыми (единицы микросекунд на шаг при времени шага 21 мкс). При p = 16 (заняты и вторые аппаратные потоки ядер) время ухудшается и приобретает огромный разброс (до 0,64 с между прогонами): два потока делят одно ядро, а рядом работает Windows. Выводы по p = 16 в режиме Б поэтому имеют низкую достоверность.

3.6. Латентность и пропускная способность MPI

Программа pingpong.c пересылает сообщения длиной от 1 байта до 64 МиБ между двумя рангами, привязанными к разным ядрам; измеряется половина времени туда-обратно (медиана 4001/501/41 повторов в зависимости от размера, первый отброшен). Результаты подогнаны по формуле (1) методом наименьших квадратов с относительными весами (чтобы малые сообщения не теряли вес).

Таблица 3 — Параметры формулы (1) для MPI через разделяемую память

Подгонка	**t_s, мкс**	**B, ГБ/с**	**M* = t_s·B**	**макс. отн. ошибка**
Все размеры (1 Б – 64 МиБ)	0.23	14.2	3.2 КиБ	75 %
Рабочий диапазон 4 КиБ – 4 МиБ	1.20	16.5	19.3 КиБ	26 %

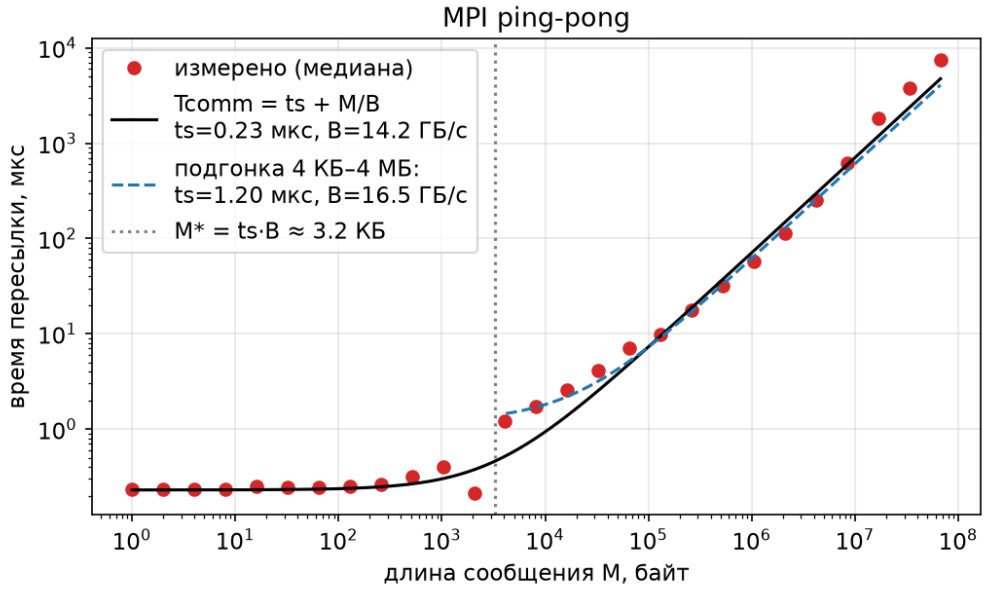


Рис. 3 — Время одностороннего сообщения и подгонка по формуле (1)

Измеренное время сообщения в 1 байт — 0.23 мкс. Члены латентности и пропускной способности равны при длине сообщения M* = t_s·B: примерно 3 КиБ по подгонке на всём диапазоне и около 19 КиБ по подгонке рабочего диапазона. Простая модель (1) здесь описывает данные лишь приближённо: Open MPI меняет протокол при переходе от коротких сообщений к длинным (около 4 КиБ — скачок времени с 0,21 до 1,2 мкс), а при размерах больше размера кэша L3 время растёт быстрее линейного (при 64 МиБ B падает до ≈ 9 ГБ/с). Поэтому приведены две подгонки, а значение M* следует читать как порядок величины — от единиц до десятков килобайт. Главный практический вывод для нашей задачи: граничное сообщение (8 байт) на 2–3 порядка короче M*, его цена ≈ 0,23 мкс — это чистая латентность, и при шаге порядка десятков микросекунд (режим Б) обмен почти не влияет на результат.

3.7. Гибрид: деление 16 аппаратных потоков

Таблица 4 — Три и более деления одного и того же ресурса (всего 16 потоков)

Процессов × потоков	Режим А: T, с	разброс	Режим Б: T, с	разброс
16 × 1	0.826	0.241	0.339	0.109
8 × 2	0.826	0.044	0.416	0.392
4 × 4	0.812	0.021	0.441	0.477
2 × 8	0.832	0.133	0.457	0.230
1 × 16	0.833	0.041	0.461	0.137

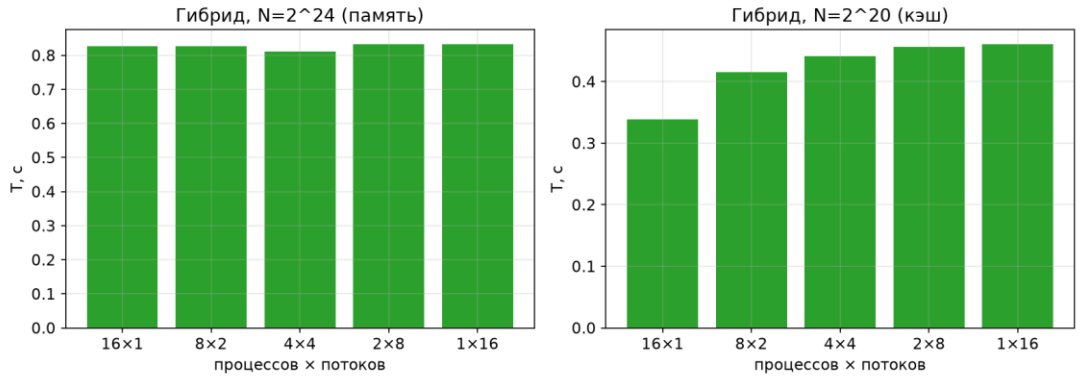


Рис. 4 — Время гибридной программы при разных делениях 16 потоков

Режим А: все деления в пределах 0,81–0,83 с, т. е. различия внутри разброса — задача ограничена шиной памяти, которой всё равно, кто к ней обращается. Режим Б: лучшее деление — 16 × 1 (чистый MPI, ≈ 0,34 с); по мере роста числа потоков внутри процесса время растёт до ≈ 0,46 с при 1 × 16. Объяснение через архитектуру узла: на нашей машине один сокет и один NUMA-домен, поэтому главные доводы за гибрид (не пересекать границу сокета, не плодить ранги на NUMA-домен) не работают. Зато проявляются издержки гибрида: схема FUNNELED заставляет все потоки ждать, пока главный поток выполнит обмен (два барьера на шаг), а 16 потоков в одном процессе включают оба аппаратных потока каждого ядра. Это означает лишь, что на одном узле без NUMA гибрид не даёт выигрыша по времени; его выигрыш по памяти показан в разделе 4.5, а выигрыш по числу рангов на многоузловом кластере здесь не измерялся.

4. Аналитическая часть

4.1. Сводная таблица сравнения моделей

Таблица 5 — Сравнение моделей по критериям из теоретической части

Критерий	OpenMP	MPI
Модель памяти и видимость данных	Единое адресное пространство: любой поток видит все данные, поэтому код короче, но корректность зависит от того, что объявлено private/shared; забытый private — молчаливая гонка.	Раздельные адресные пространства: нужные данные видны только после явной передачи; граница между «моим» и «чужим» всегда на виду, что дисциплинирует декомпозицию.
Механизм и стоимость коммуникации	Неявный, через кэш: чтение чужих данных бесплатно синтаксически; цена — барьеры и трафик когерентности (на нашем стенде барьер на 8 потоках укладывается в единицы мкс на шаг).	Явные сообщения: цена каждого — t_s + M/B (у нас 0,23 мкс для 8 байт и ≈ 16 ГБ/с для крупных), поэтому мелкие обмены нужно укрупнять.
Предпочитаемая гранулярность	Мелкая и средняя: параллельный цикл со сотнями-тысячами итераций на поток; fork-join выгоден, пока тело цикла дороже барьера.	Крупная: большие подобласти, обмен реже и крупнее; мелкие задачи тонут в накладных расходах сообщений.
Предел масштабируемости и его причина	Один узел (число ядер и пропускная способность его памяти): у нас S ≈ 1,5 в режиме А на 8 потоках — потолок задаёт шина памяти, а не язык.	Практически не ограничен моделью: растёт с числом узлов, пока не упрётся в сеть, балансировку и коллективные операции; на одном узле MPI упирается в ту же шину памяти (S ≈ 1,4).
Инкрементальное распараллеливание	Да: добавили директиву к одному циклу — остальное работает как раньше; без -fopenmp программа остаётся корректной последовательной.	Нет: уже на первом шаге нужно решить, кто что хранит; последовательной версии «по пути» не существует.
Отладка и классы дефектов	Один процесс — привычные отладчик и профилировщик; но дефекты недетерминированные: гонки, ложное разделение, взаимные блокировки замков; воспроизвести трудно.	Дефекты другого рода: взаимная блокировка из-за порядка Send/Recv, несовпадение тегов и размеров; зато они обычно воспроизводятся и видны в трассе; отладка N процессов неудобна.
Переносимость между машинами и компиляторами	Стандарт переносим, но поведение зависит от компилятора и платформы (привязка потоков, поддержка версий 5.x, размещение памяти); производительность переносится хуже корректности.	Корректная программа работает от ноутбука до суперкомпьютера без изменения кода; различия — в реализации MPI и сети; производительность тоже требует настройки, но модель одна.

4.2. Преимущества OpenMP (с данными измерений)

- Инкрементальность: параллельная версия отличается от последовательной одной директивой/блоком; в нашем коде OpenMP-вариант (solver_omp.c) — самый короткий и не содержит ни одного вызова обмена.
- Отсутствие явного перемещения данных: в OpenMP-программе нет ни halo-обмена, ни сборки результата. На малом числе потоков это даёт то же время, что и MPI (p = 4: 0.880 с против 0.901 с в режиме А).

- Дешёвая синхронизация внутри сокета: в режиме Б при p = 4 OpenMP показывает S = 3.13 (E = 0.78), т. е. барьер на 4 потоках почти не заметен на фоне шага ≈ 27 мкс.
- Расход памяти: 1 процесс × 16 потоков занимает 2,1 МиБ при N = 2^16 против 248 МиБ у 16 MPI-процессов (раздел 4.5).
- Простота отладки: один процесс, обычный gdb/профилировщик; результат побитово совпал с эталоном при всех p (раздел 3.4).

4.3. Преимущества MPI (с данными измерений)

- Масштабируемость за пределы одного узла: единственная из двух моделей, способная использовать несколько машин. На нашем одноузловом стенде это измерить нельзя — приведён лишь теоретический довод; измеренная часть показывает, что на одном узле MPI не медленнее OpenMP (режим Б, p = 8: 0.165 с против 0.164 с).
- Явная локальность: MPI-программа с первой строки хранит данные в локальных массивах размером N/p — это то, что на NUMA-машине OpenMP-программисту приходится добиваться отдельно (first touch, привязка потоков). В режиме Б при p = 8 локальный набор 0,5 МиБ на ранг помещается в L2 — это объясняет лучшую эффективность.
- Отсутствие когерентности между процессами: ложное разделение между рангами невозможно. Для сравнения, в OpenMP ложное разделение в нашем тесте falseshare.c замедлило программу в 1.9 раза при 4 потоках (0.106 с против 0.055 с)* — это скромный результат из-за простоты теста, но направление эффекта подтверждено.
- Переносимость корректности: один и тот же текст работал при R = 1, 2, 4, 8, 16 без изменений и давал идентичный результат.

* Тест использует volatile-счётчики и -O1; в реальных приложениях (общие массивы сумм, счётчики, редукции вручную) замедление бывает кратно больше. Многократный разброс этих чисел между запусками (1,15–1,94) говорит о том, что измерение шумное.

4.4. Ограничения каждой модели

OpenMP. (1) Один узел: потоки не могут выйти за пределы общей памяти, что подтверждено на практике потолком ускорения по шине памяти. (2) Ложное разделение: эффект измерен выше (до ×1,9). (3) Зависимость от first touch в NUMA-системах: страницы размещаются рядом с потоком, который первым к ним обратился; поэтому инициализация в наших программах выполняется параллельно. На стенде с одним NUMA-узлом эффект измерить нельзя — это прямо отмечено как ограничение эксперимента. (4) Сложность рассуждений о гонках: корректность не следует из локального чтения кода.

MPI. (1) Цена каждого обмена: от 0,23 мкс для 8 байт до ≈ 7,4 мс для 64 МиБ; нужно проектировать так, чтобы обмен был редким и крупным. (2) Невозможность инкрементального внедрения: вся программа переписывается вокруг декомпозиции. (3) Дублирование памяти по процессам: см. 4.5. (4) Сложность кода: для той же схемы в MPI-варианте добавились расчёт смещений, призрачные ячейки, Sendrecv, Gatherv для проверки.

4.5. Когда оправдан гибрид; память

Таблица 6 — Суммарный пик резидентной памяти (RSS) по всем процессам, МиБ

N	данные, МиБ	1 × 16 потоков	2 × 8	16 процессов	Разница MPI−OMP, МиБ	MPI больше на
2^16	1	2.1	33.8	248.2	246	11719 %
2^20	16	17.7	48.6	260.3	243	1371 %
2^24	256	258.4	285.6	500.9	242	94 %
2^26	1024	1026.6	1053.9	1270.8	244	24 %

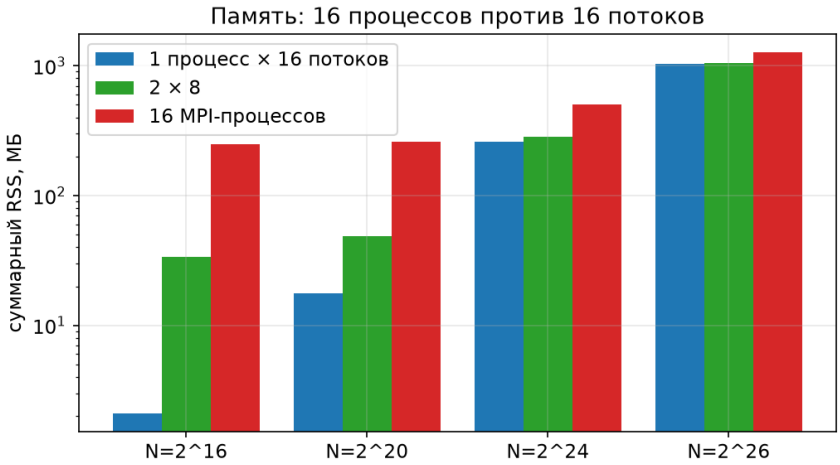


Рис. 5 — Память: 16 MPI-процессов против 16 потоков

Разница между 16 MPI-процессами и одним процессом с 16 потоками почти постоянна — около 244 МиБ, т. е. ≈ 15 МиБ на каждый процесс (среда выполнения MPI, библиотеки, служебные буферы). Сами данные не дублируются: декомпозиция по областям делит их без повторов, поэтому дублирование проявляется только как фиксированная цена процесса. Эта цена становится решающей, когда суммарный объём данных узла сопоставим с 16 × 15 ≈ 244 МиБ: при N = 2^16 MPI-версия занимает в 118 раз больше, при N = 2^24 — на 94 % больше, при N = 2^26 — всего на 24 %. Порог «решающей» разницы по порядку величины — рабочий набор около 0,25–1 ГиБ на узел (для нашей задачи N ≈ 2^24–2^26); для реальных приложений с реплицируемыми таблицами, сетками или буферами каждый процесс хранит копию, и разница растёт пропорционально числу процессов — это основной довод за гибрид по памяти. Гибрид 2 × 8 занимает промежуточное место (+31 МиБ при N = 2^16, т. е. ≈ 16 МиБ на процесс).

Условия, при которых гибрид оправдан: (а) узел содержит несколько сокетов/NUMA-доменов — тогда ранг на домен исключает межсокетный трафик и обеспечивает first touch; (б) данные реплицируются по рангам и память узла ограничена; (в) число рангов настолько велико, что накладные расходы коллективных операций и сообщений растут; (г) задача допускает мелкозернистый параллелизм внутри домена. На нашем одноузловом одно-NUMA стенде условия (а) и (в) не выполняются, и по времени гибрид выигрыша не дал (раздел 3.7) — это тоже результат.

5. Заключение

Выбор модели. Для стационарного шаблонного расчёта вроде нашего, размером до ≈ 1 ГиБ на одном узле, я бы выбрал OpenMP. Основания — измеренные цифры: время в режиме Б при p = 8 практически одинаково (0.164 с у OpenMP и 0.165 с у MPI), в режиме А OpenMP не хуже (p = 8: 0.760 с против 0.830 с), а код не содержит обмена и при малых N экономит сотни мегабайт памяти. Для задачи, которая не помещается в один узел (или должна работать на кластере), выбор — MPI, а при наличии нескольких NUMA-доменов или ограничения по памяти — гибрид с одним рангом на домен. Это рекомендация по принципу «используйте модель, соответствующую памяти машины», подкреплённая измерениями только для одного узла; многоузловое поведение в работе не измерялось и описано как теоретический вывод.

Главный наблюдаемый факт: на одном узле для данной задачи различия между моделями меньше, чем различия, вносимые железом — пропускной способностью памяти (потолок S ≈ 1,5 в режиме А) и использованием SMT (деградация при p = 16). Выбор модели — вопрос не «какая быстрее», а «какой памяти она соответствует».

Перспективы на узлах с ускорителями. Появляется третий уровень параллелизма: MPI между узлами, потоки (OpenMP) на ядрах CPU, и ядра графического ускорителя (CUDA/HIP, OpenMP target offload, SYCL). Модель памяти усложняется: у ускорителя своя память, обмен

«хост ↔ устройство» и «устройство ↔ устройство через сеть» становятся главным узким местом. Поэтому ожидаемо сохранение MPI как межузлового уровня (с передачей данных напрямую из памяти устройства при поддержке GPU-aware MPI), а роль OpenMP в части кода сводится к управлению хостом и описанию переноса работы на устройство директивами target. Гибридные программы вида «MPI + X» [6] становятся не исключением, а нормой; число уровней растёт, и тем важнее измерять, а не предполагать.

Ограничения работы: один узел, WSL2 с шумом от хост-системы; транспорт MPI — разделяемая память; не измерены NUMA/first touch и многоузловой режим; тест ложного разделения упрощён. Эти ограничения не отменяют выводов раздела 4, но определяют границы их применимости.

Список литературы

[1] Pacheco P., Malensek M. An Introduction to Parallel Programming. 2nd ed. Morgan Kaufmann, 2021.

[2] Rauber T., Rünger G. Parallel Programming for Multicore and Cluster Systems. 3rd ed. Springer, 2023.

[3] Quinn M. J. Parallel Programming in C with MPI and OpenMP. McGraw-Hill, 2022 (в редакции, указанной в списке рекомендованной литературы задания).

[4] MPI: A Message-Passing Interface Standard, version 4.1 and later. MPI Forum. https://www.mpi-forum.org

[5] OpenMP Application Programming Interface, version 5.2 and later. OpenMP ARB. https://www.openmp.org

[6] HLRS self-study materials on hybrid programming, MPI + X. https://www.hlrs.de/training/self-study-materials

[7] Gropp W., Lusk E., Skjellum A. Using MPI: Portable Parallel Programming with the Message-Passing Interface. 3rd ed. MIT Press, 2014.

[8] Chapman B., Jost G., van der Pas R. Using OpenMP: Portable Shared Memory Parallel Programming. MIT Press, 2008.

Приложение А. Исходный код (не входит в объём 8–10 страниц)

src/core.h

```
/* core.h - общее численное ядро для всех трёх реализаций (SIW 2).
 * Уравнение теплопроводности u_t = a*u_xx, явная схема FTCS, r = a*dt/dx^2 = 0.25 (устойчиво при r<=0.5).
 * Массив с «призрачными» ячейками: u[0] и u[N+1] - границы Дирихле (=0), u[1..N] - внутренние узлы. */
#ifndef CORE_H
#define CORE_H
#include <math.h>
#include <stdlib.h>
#include <sys/resource.h>


#define R_COEF 0.25


/* начальное условие для глобального узла i (0..N-1) */
static inline double init_val(long i, long N) {
    double x = (double)(i + 1) / (double)(N + 1);
    return sin(M_PI * x) + 0.5 * sin(3.0 * M_PI * x);
}


/* ОДИН узел схемы - одно и то же выражение во всех реализациях */
static inline double heat_point(const double *u, long i) {
    return u[i] + R_COEF * (u[i - 1] - 2.0 * u[i] + u[i + 1]);
}


/* Ядро: обновление диапазона [lo,hi) - одна и та же функция во всех реализациях (restrict даёт компилятору право
векторизовать) */
static inline void kernel(const double *restrict a, double *restrict b, long lo, long hi) {
    for (long i = lo; i < hi; i++) b[i] = heat_point(a, i);
}


/* последовательный эталон: возвращает массив длины N+2 (результат после steps шагов) */
static double *solve_serial(long N, long steps) {
    double *u = calloc(N + 2, sizeof(double)), *v = calloc(N + 2, sizeof(double));
    for (long i = 0; i < N; i++) u[i + 1] = init_val(i, N);
    for (long s = 0; s < steps; s++) {
        kernel(u, v, 1, N + 1);
        double *t = u; u = v; v = t;
    }
    free(v);
    return u;
}


static int cmpd(const void *a, const void *b) {
    double x = *(const double *)a, y = *(const double *)b;
    return (x > y) - (x < y);
}
/* медиана после отбрасывания первого прогона */
static double median_skip_first(double *t, int reps) {
    int n = reps - 1;
    double *c = malloc(n * sizeof(double));
    for (int i = 1; i < reps; i++) c[i - 1] = t[i];
    qsort(c, n, sizeof(double), cmpd);
    double m = (n % 2) ? c[n / 2] : 0.5 * (c[n / 2 - 1] + c[n / 2]);
    free(c);
    return m;
}
static double minv(double *t, int n) { double m = t[1]; for (int i = 1; i < n; i++) if (t[i] < m) m = t[i]; return m;
}
static double maxv(double *t, int n) { double m = t[1]; for (int i = 1; i < n; i++) if (t[i] > m) m = t[i]; return m;
}
static double maxrss_mb(void) { struct rusage ru; getrusage(RUSAGE_SELF, &ru); return ru.ru_maxrss / 1024.0; }
#endif
```

src/solver_omp.c

```
/* solver_omp.c - реализация на OpenMP (общая память, fork-join).
 * gcc -O3 -g -fopenmp solver_omp.c -o solver_omp -lm ;  ./solver_omp N steps reps verify */
#include <stdio.h>
#include <omp.h>
#include "core.h"


static double *run(long N, long steps, double *time_out) {
    double *u = malloc((N + 2) * sizeof(double)), *v = malloc((N + 2) * sizeof(double));
    /* «первое касание» - параллельная инициализация, страницы размещаются рядом с потоками */
    #pragma omp parallel for schedule(static)
    for (long i = 0; i < N + 2; i++) {
```

```
        u[i] = (i == 0 || i == N + 1) ? 0.0 : init_val(i - 1, N);
        v[i] = 0.0;
    }
    double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num(), nt = omp_get_num_threads();
        double *a = u, *b = v;           /* у каждого потока свои указатели; меняются синхронно */
        for (long s = 0; s < steps; s++) {
            /* статическое разбиение work-sharing: поток tid обновляет свой блок */
            long lo = 1 + N * tid / nt, hi = 1 + N * (tid + 1) / nt;
            kernel(a, b, lo, hi);
            #pragma omp barrier
            double *t = a; a = b; b = t;
        }
    }
    *time_out = omp_get_wtime() - t0;
    double *res = (steps % 2) ? v : u, *other = (steps % 2) ? u : v;
    free(other);
    return res;
}

int main(int argc, char **argv) {
    long N = argc > 1 ? atol(argv[1]) : (1L << 24);
    long steps = argc > 2 ? atol(argv[2]) : 100;
    int reps = argc > 3 ? atoi(argv[3]) : 6;
    int verify = argc > 4 ? atoi(argv[4]) : 0;
    int p = omp_get_max_threads();
    if (verify) {
        double t; double *r = run(N, steps, &t), *ref = solve_serial(N, steps), d = 0;
        for (long i = 1; i <= N; i++) { double e = fabs(r[i] - ref[i]); if (e > d) d = e; }
        printf("VERIFY impl=omp N=%ld steps=%ld p=%d linf=%.3e\n", N, steps, p, d);
        return 0;
    }
    double *times = malloc(reps * sizeof(double));
    for (int k = 0; k < reps; k++) { double *r = run(N, steps, &times[k]); free(r); }
    printf("RESULT impl=omp N=%ld steps=%ld procs=1 threads=%d median=%.6f min=%.6f max=%.6f rss_mb=%.1f\n",
           N, steps, p, median_skip_first(times, reps), minv(times, reps), maxv(times, reps), maxrss_mb());
    return 0;
}
```

src/solver_mpi.c

```
/* solver_mpi.c - реализация на MPI (раздельная память, SPMD), разбиение области на полосы.
 * mpicc -O3 -g solver_mpi.c -o solver_mpi -lm ;  mpirun -n 8 ./solver_mpi N steps reps verify
 * Если скомпилировано с -DHYBRID -fopenmp - гибрид MPI+OpenMP (solver_hyb). */
#include <stdio.h>
#include <mpi.h>
#ifdef HYBRID
#include <omp.h>
#endif
#include "core.h"

static void halo(double *a, long n, int rank, int size, MPI_Comm c) {
    int left = rank > 0 ? rank - 1 : MPI_PROC_NULL, right = rank < size - 1 ? rank + 1 : MPI_PROC_NULL;
    MPI_Sendrecv(&a[1], 1, MPI_DOUBLE, left, 0, &a[n + 1], 1, MPI_DOUBLE, right, 0, c, MPI_STATUS_IGNORE);
    MPI_Sendrecv(&a[n], 1, MPI_DOUBLE, right, 1, &a[0], 1, MPI_DOUBLE, left, 1, c, MPI_STATUS_IGNORE);
}

static double *run(long n, long off, long N, long steps, int rank, int size, double *time_out) {
    double *u = malloc((n + 2) * sizeof(double)), *v = malloc((n + 2) * sizeof(double));
#ifdef HYBRID
    #pragma omp parallel for schedule(static)
#endif
    for (long i = 0; i < n + 2; i++) {
        long g = off + i - 1;                          /* глобальный индекс */
        u[i] = (g < 0 || g >= N) ? 0.0 : init_val(g, N);
        v[i] = 0.0;
    }
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();
#ifdef HYBRID
    #pragma omp parallel
    {
        int tid = omp_get_thread_num(), nt = omp_get_num_threads();
        double *a = u, *b = v;
        for (long s = 0; s < steps; s++) {
            #pragma omp master
            halo(a, n, rank, size, MPI_COMM_WORLD);        /* MPI_THREAD_FUNNELED: вызывает только master */
            #pragma omp barrier
            { long lo = 1 + n * tid / nt, hi = 1 + n * (tid + 1) / nt; kernel(a, b, lo, hi); }
            #pragma omp barrier
```

```
            double *t = a; a = b; b = t;
        }
    }
#else
    {
        double *a = u, *b = v;
        for (long s = 0; s < steps; s++) {
            halo(a, n, rank, size, MPI_COMM_WORLD);
            kernel(a, b, 1, n + 1);
            double *t = a; a = b; b = t;
        }
    }
#endif
    MPI_Barrier(MPI_COMM_WORLD);
    double dt = MPI_Wtime() - t0, mx;
    MPI_Allreduce(&dt, &mx, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
    *time_out = mx;
    double *res = (steps % 2) ? v : u, *other = (steps % 2) ? u : v;
    free(other);
    return res;
}

int main(int argc, char **argv) {
    int rank, size, provided = 0;
#ifdef HYBRID
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    const char *impl = "hyb"; int thr = omp_get_max_threads();
#else
    MPI_Init(&argc, &argv);
    const char *impl = "mpi"; int thr = 1;
#endif
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); MPI_Comm_size(MPI_COMM_WORLD, &size);
    long N = argc > 1 ? atol(argv[1]) : (1L << 24);
    long steps = argc > 2 ? atol(argv[2]) : 100;
    int reps = argc > 3 ? atoi(argv[3]) : 6;
    int verify = argc > 4 ? atoi(argv[4]) : 0;
    long base = N / size, rem = N % size;
    long n = base + (rank < rem ? 1 : 0), off = rank * base + (rank < rem ? rank : rem);

    if (verify) {
        double t; double *r = run(n, off, N, steps, rank, size, &t);
        int *cnt = NULL, *dsp = NULL; double *all = NULL;
        if (rank == 0) { cnt = malloc(size * sizeof(int)); dsp = malloc(size * sizeof(int)); all = malloc(N *
sizeof(double)); }
        int ni = (int)n, oi = (int)off;
        MPI_Gather(&ni, 1, MPI_INT, cnt, 1, MPI_INT, 0, MPI_COMM_WORLD);
        MPI_Gather(&oi, 1, MPI_INT, dsp, 1, MPI_INT, 0, MPI_COMM_WORLD);
        MPI_Gatherv(&r[1], ni, MPI_DOUBLE, all, cnt, dsp, MPI_DOUBLE, 0, MPI_COMM_WORLD);
        if (rank == 0) {
            double *ref = solve_serial(N, steps), d = 0;
            for (long i = 0; i < N; i++) { double e = fabs(all[i] - ref[i + 1]); if (e > d) d = e; }
            printf("VERIFY impl=%s N=%ld steps=%ld procs=%d threads=%d linf=%.3e\n", impl, N, steps, size, thr, d);
        }
        MPI_Finalize(); return 0;
    }
    double *times = malloc(reps * sizeof(double));
    for (int k = 0; k < reps; k++) { double *r = run(n, off, N, steps, rank, size, &times[k]); free(r); }
    double rss = maxrss_mb(), tot;
    MPI_Reduce(&rss, &tot, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0)
        printf("RESULT impl=%s N=%ld steps=%ld procs=%d threads=%d median=%.6f min=%.6f max=%.6f rss_mb=%.1f\n",
               impl, N, steps, size, thr, median_skip_first(times, reps), minv(times, reps), maxv(times, reps), tot);
    MPI_Finalize();
    return 0;
}
```

src/pingpong.c

```
/* pingpong.c - измерение времени пересылки T(M) между двумя рангами для подгонки Tcomm = ts + M/B.
 * mpirun -n 2 ./pingpong  -> строки "PP bytes=.. time_us=.." (одностороннее время, медиана) */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
static int cmpd(const void *a, const void *b) { double x = *(double *)a, y = *(double *)b; return (x > y) - (x < y); }
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank; MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    long maxb = 1L << 26; char *buf = calloc(maxb, 1);
    for (long m = 1; m <= maxb; m *= 2) {
        int reps = m <= 4096 ? 4001 : (m <= (1 << 20) ? 501 : 41);
        double *t = malloc(reps * sizeof(double));
        for (int k = 0; k < reps; k++) {
```

```
            MPI_Barrier(MPI_COMM_WORLD);
            double t0 = MPI_Wtime();
            if (rank == 0) { MPI_Send(buf, (int)m, MPI_BYTE, 1, 0, MPI_COMM_WORLD); MPI_Recv(buf, (int)m, MPI_BYTE, 1,
0, MPI_COMM_WORLD, MPI_STATUS_IGNORE); }
            else           { MPI_Recv(buf, (int)m, MPI_BYTE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE); MPI_Send(buf,
(int)m, MPI_BYTE, 0, 0, MPI_COMM_WORLD); }
            t[k] = (MPI_Wtime() - t0) / 2.0;
        }
        qsort(t + 1, reps - 1, sizeof(double), cmpd);   /* первый прогон отбрасываем */
        if (rank == 0) printf("PP bytes=%ld time_us=%.4f\n", m, t[1 + (reps - 1) / 2] * 1e6);
        free(t);
    }
    MPI_Finalize(); return 0;
}
```

src/falseshare.c

```
/* falseshare.c - демонстрация ложного разделения (false sharing) в OpenMP.
 * Каждый поток увеличивает СВОЙ счётчик: (а) счётчики лежат рядом в одной строке кэша, (б) разнесены на 64 байта. */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#define ITER 200000000L
int main(void) {
    int p = omp_get_max_threads();
    volatile long *packed = calloc(p, sizeof(long));
    volatile long *padded = calloc(p * 8, sizeof(long));
    double t0 = omp_get_wtime();
    #pragma omp parallel
    { int id = omp_get_thread_num(); for (long i = 0; i < ITER; i++) packed[id]++; }
    double ta = omp_get_wtime() - t0; t0 = omp_get_wtime();
    #pragma omp parallel
    { int id = omp_get_thread_num(); for (long i = 0; i < ITER; i++) padded[id * 8]++; }
    double tb = omp_get_wtime() - t0;
    printf("FALSESHARE threads=%d packed=%.3f padded=%.3f ratio=%.2f\n", p, ta, tb, ta / tb);
    return 0;
}
```

scripts/build.sh

```
#!/bin/bash
# Сборка всех программ (запускать в WSL/Linux). Результат - в ./bin
set -e
cd "$(dirname "$0")/../src"; mkdir -p ../bin
gcc   -O3 -g -fopenmp solver_omp.c -o ../bin/solver_omp -lm
mpicc -O3 -g solver_mpi.c -o ../bin/solver_mpi -lm
mpicc -O3 -g -fopenmp -DHYBRID solver_mpi.c -o ../bin/solver_hyb -lm
mpicc -O3 -g pingpong.c -o ../bin/pingpong
gcc   -O1 -g -fopenmp falseshare.c -o ../bin/falseshare
echo build ok
```

scripts/run_all.sh

```
#!/bin/bash
# Полный набор измерений SIW 2. Запускать в WSL/Linux после build.sh.  Результат: results/*.txt
# Размещение задаётся явно: каждому MPI-процессу выделяется непрерывный набор из 16/R аппаратных потоков,
# потоки OpenMP внутри набора раскладываются OMP_PROC_BIND=spread, OMP_PLACES=threads.
cd "$(dirname "$0")/.."
export OMPI_ALLOW_RUN_AS_ROOT=1 OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1
export OMP_PLACES=threads OMP_PROC_BIND=spread
mkdir -p results
N=${N:-16777216}; S=${S:-100}; REPS=${REPS:-6}
MPIRUN() { local R=$1; shift; mpirun -n $R --use-hwthread-cpus --map-by ppr:$R:node:PE=$((16/R)) --bind-to hwthread
"$@"; }


echo "# $(date) N=$N steps=$S reps=$REPS" > results/scaling.txt
for p in 1 2 4 8 16; do
  OMP_NUM_THREADS=$p ./bin/solver_omp $N $S $REPS >> results/scaling.txt
  OMP_NUM_THREADS=1 MPIRUN $p ./bin/solver_mpi $N $S $REPS >> results/scaling.txt
  if [ $p -eq 1 ]; then R=1; T=1; else R=2; T=$((p/2)); fi
  OMP_NUM_THREADS=$T MPIRUN $R ./bin/solver_hyb $N $S $REPS >> results/scaling.txt
done


echo "# hybrid divisions, total 16 hardware threads" > results/hybrid.txt
for cfg in "16 1" "8 2" "4 4" "2 8" "1 16"; do
  set -- $cfg
  OMP_NUM_THREADS=$2 MPIRUN $1 ./bin/solver_hyb $N $S $REPS >> results/hybrid.txt
done


echo "# memory: 16 MPI processes vs 1 process x 16 threads vs hybrid 2x8" > results/memory.txt
for n in 65536 1048576 16777216 67108864; do
```

```
  OMP_NUM_THREADS=16 ./bin/solver_omp $n 4 3 >> results/memory.txt
  OMP_NUM_THREADS=1 MPIRUN 16 ./bin/solver_mpi $n 4 3 >> results/memory.txt
  OMP_NUM_THREADS=8 MPIRUN 2 ./bin/solver_hyb $n 4 3 >> results/memory.txt
done


echo "# pingpong (2 ranks on different cores, shared-memory transport)" > results/pingpong.txt
mpirun -n 2 --use-hwthread-cpus --map-by ppr:2:node:PE=8 --bind-to hwthread ./bin/pingpong >> results/pingpong.txt


echo "# false sharing" > results/falseshare.txt
for t in 2 4 8; do OMP_NUM_THREADS=$t ./bin/falseshare >> results/falseshare.txt; done


echo "# system" > results/system.txt
lscpu | grep -E "Model name|^CPU\(s\)|Thread|Core|Socket|L1d|L2|L3|NUMA node\(s\)" >> results/system.txt
free -m | head -2 >> results/system.txt
mpirun --version | head -1 >> results/system.txt
gcc --version | head -1 >> results/system.txt
echo done
```

scripts/run_cache.sh

```
#!/bin/bash
# Второй набор: задача целиком помещается в кэш (N=2^18, 4 МБ), 8000 шагов - упор в синхронизацию, а не в память.
cd "$(dirname "$0")/.."
export OMPI_ALLOW_RUN_AS_ROOT=1 OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1
export OMP_PLACES=threads OMP_PROC_BIND=spread
N=262144; S=8000; REPS=11
MPIRUN() { local R=$1; shift; mpirun -n $R --use-hwthread-cpus --map-by ppr:$R:node:PE=$((16/R)) --bind-to hwthread
"$@"; }
echo "# cache-resident: N=$N steps=$S reps=$REPS" > results/scaling_cache.txt
for p in 1 2 4 8 16; do
  OMP_NUM_THREADS=$p ./bin/solver_omp $N $S $REPS >> results/scaling_cache.txt
  OMP_NUM_THREADS=1 MPIRUN $p ./bin/solver_mpi $N $S $REPS >> results/scaling_cache.txt
  if [ $p -eq 1 ]; then R=1; T=1; else R=2; T=$((p/2)); fi
  OMP_NUM_THREADS=$T MPIRUN $R ./bin/solver_hyb $N $S $REPS >> results/scaling_cache.txt
done
echo "# hybrid divisions, cache-resident" > results/hybrid_cache.txt
for cfg in "16 1" "8 2" "4 4" "2 8" "1 16"; do
  set -- $cfg
  OMP_NUM_THREADS=$2 MPIRUN $1 ./bin/solver_hyb $N $S $REPS >> results/hybrid_cache.txt
done
echo done
```

scripts/verify.sh

```
#!/bin/bash
# Проверка корректности: L_inf-норма разности с последовательной версией (N не кратно числу процессов намеренно)
cd "$(dirname "$0")/.."
export OMPI_ALLOW_RUN_AS_ROOT=1 OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1 OMPI_MCA_rmaps_base_oversubscribe=1
N=1000003; S=200
for p in 1 2 4 8 16; do
  OMP_NUM_THREADS=$p ./bin/solver_omp $N $S 6 1
done
for p in 1 2 4 8 16; do
  mpirun -n $p --use-hwthread-cpus ./bin/solver_mpi $N $S 6 1
done
for cfg in "16 1" "8 2" "4 4" "2 8" "1 16"; do
  set -- $cfg
  OMP_NUM_THREADS=$2 mpirun -n $1 --use-hwthread-cpus ./bin/solver_hyb $N $S 6 1
done
```